

Closing the Operational Gap in Semantic Caching

Aditeya Baral* / Radoslav Ralev* / Iliya Sotirov Zhechev / Srijith Rajamohan / Jen Agarwal
New York University · Redis



Paper & code
github.com/aditeyabaral/
operational-gap-semantic-caching
 Models & datasets
huggingface.co/redis
 Evals & infra
redis.io/langcache



PR-AUC P-CHR AUC | Model selection for semantic caching is a **threshold-utility** problem, not a ranking one.

1 The mismatch

A semantic cache serves a stored LLM response when a query's similarity score clears a fixed threshold τ . Deployment is a hard binary decision on a score *magnitude*. Yet the field evaluates them with **PR-AUC**, which measures only how well scores *rank* — not where on the scale they sit.

A model can rank every positive above every negative yet collapse to near-chance precision once the cache fires at a useful rate

Threshold utility, not probability calibration. A score can be poorly calibrated yet perfectly usable at a threshold. What deployment needs is a measure that holds across models with different score semantics, and PR-AUC is not one.

2 Cache-aware metrics

P-CHR AUC
Precision of the deployed top-1 decision, against **cache hit ratio** as τ is swept — the fraction of queries answered from cache, and the driver of cost savings.

$$\text{P-CHR AUC} = \int_0^1 \text{Precision}(\text{CHR}^{-1}(c)) \, dc$$

Operational Retention Rate
The share of **offline ranking quality** that survives at deployment.

$$\text{ORR} = \frac{\text{P-CHR AUC}}{\text{PR-AUC}}$$

The floor depends on the duplicate rate, so absolute P-CHR AUC does not transfer across deployments, but which training objective deploys best transfers unchanged.

3 Decomposing the gap

The measured **operational gap**
 $\Delta_{\text{op}} = \text{PR-AUC} - \text{P-CHR AUC}$ splits in two.

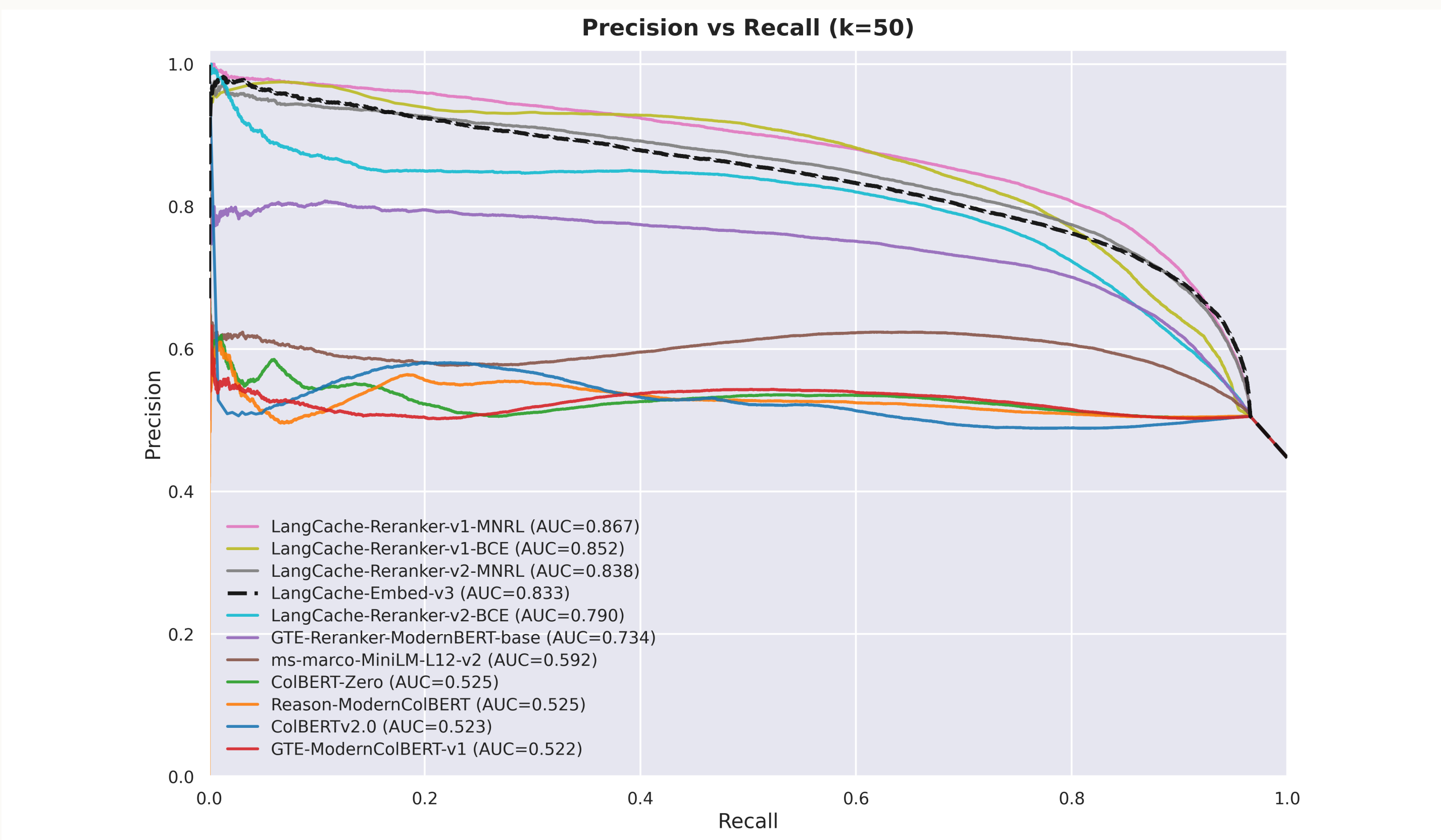
Δ_{str} **Structural.** $1 - p(1 - \ln p)$, fixed by the positive rate alone. Irreducible.

Δ_{util} **Threshold utility.** $\max(0, \Delta_{\text{op}} - \Delta_{\text{str}})$, recoverable loss from where a model *places* its scores.

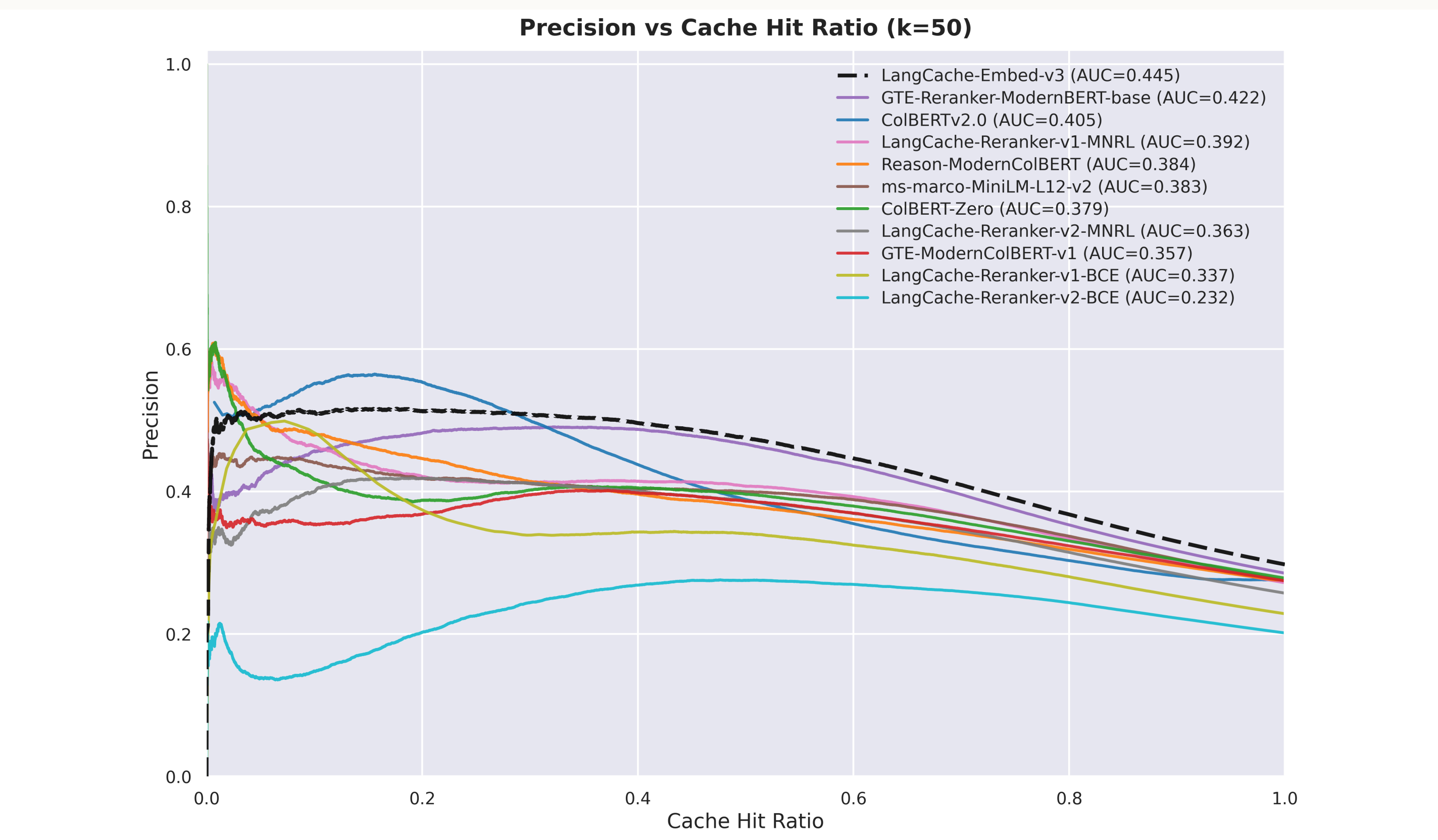
Rescaling invariant. P-CHR AUC is unchanged by any monotone rescaling, temperature or Platt included, so it compares models with different score semantics and cannot be improved by post-hoc calibration.

What does close it is normalizing scores over the candidate pool, which doubles as a training-free deployment fix.

4 The same ten rerankers, two views paired with LangCache-Embed-v3 at K = 50 · 74,265 test pairs, 45% positive



Offline. Every model looks healthy; BCE sits near the top and the ColBERT family last.



Deployed. The ranking inverts. BCE collapses, ColBERT climbs, and no reranker beats the retriever.

5 Results

Reranker	PR	P-CHR	Δ_{util}	ORR
COLBERT FAMILY				
ColBERTv2.0	0.515	0.403	0	0.782
GTE-ModernColBERT-v1	0.517	0.354	0	0.685
GENERAL CROSS-ENCODERS				
GTE-Reranker-ModernBERT	0.712	0.390	0.129	0.548
LANGCACHE RERANKERS — MNRL				
v1-MNRL · 1M	0.824	0.353	0.278	0.429
v2-MNRL · 40M	0.804	0.330	0.281	0.410
LANGCACHE RERANKERS — BCE				
v1-BCE · 1M	0.816	0.290	0.333	0.356
v2-BCE · 40M	0.748	0.171	0.384	0.229
Retriever baseline (mean)	0.685	0.383	0.109	0.562
Strongest retriever (Embed-v3) · 8M	0.833	0.445	0.195	0.534

Reranker rows are means over the 9 retrievers; selected rows only; 1M / 8M / 40M = training pairs. **Bold** marks the best reranker in each column; shaded retriever rows are for comparison. Only ColBERTv2.0 and GTE-Reranker clear the average baseline.

6 We release

All datasets and models below are curated and trained specifically for semantic caching, and ship with **Redis LangCache**.

LangCache SentencePairs. Sentence-pair corpora at three scales (1M / 8M / 40M) with well-defined train, validation and test splits.

LangCache-Embed-v3. A 22.6M-parameter MiniLM bi-encoder trained with an ArcFace objective; best deployment quality in the study.

LangCache Rerankers. Four 150M-parameter ModernBERT cross-encoders, BCE and MNRL at two training scales.

7 In practice

- Select and monitor cache models on P-CHR AUC or ORR, never PR-AUC.
- Prefer objectives whose scores spread naturally, contrastive or multi-vector, over binary cross-entropy.
- Tune τ on traffic whose duplicate rate matches deployment, and re-estimate it as that rate drifts.